

ReconForge Integration Testing Guide

How to validate full CLI flows with mocked external tools, ensuring modules produce correct output structures, findings, loot, and notes without requiring live targets or installed tools.

Overview

What is Integration Testing in ReconForge Context

Integration testing validates that modules work end-to-end: from CLI argument parsing, through phase execution, tool invocation, output parsing, findings generation, loot collection, and output file creation — all without hitting real targets.

ReconForge wraps external tools (nmap, gobuster, nuclei, etc.) that aren't available in CI/CD environments and should never be run against real targets in tests. Integration tests mock these tool invocations while validating everything else.

Why Mock External Tools

1. **External tools may not be installed** in CI/CD environments
2. **Real scans require live targets** and network access
3. **Scans take minutes to hours** — tests must complete in seconds
4. **Reproducibility** — mocked output produces deterministic results
5. **Safety** — no accidental scans against unintended targets

What to Test

- Module initialization with valid and invalid arguments
 - Phase selection and ordering
 - OPSEC mode enforcement (techniques blocked/allowed)
 - Tool wrapper command construction
 - Parser handling of realistic tool output
 - Finding generation with correct severity/confidence
 - Loot extraction and deduplication
 - Notes timeline generation
 - Output directory structure and file creation
 - Workflow orchestration and conditional branching
 - Engagement lifecycle (start, pause, resume, complete)
 - Error handling (missing tools, timeouts, parse failures)
-

Full CLI Flow Validation

How to Test End-to-End Module Execution

The key is to mock `Runner.run()` — the single point where all external commands execute. By intercepting this method, you control exactly what every tool “returns” without executing anything.

```
import json
from pathlib import Path
from unittest.mock import patch, MagicMock
from core.runner import RunResult

def make_run_result(stdout="", stderr="", returncode=0, success=True):
    """Helper to create a mock RunResult."""
    return RunResult(
        command="mocked",
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        duration=0.5,
        success=success,
    )
```

Mocking External Tools

The `Runner.run()` method accepts a command (string or list) and returns a `RunResult`. Mock it to return realistic tool output:

```
# Mock nmap output for a SYN scan
MOCK_NMAP_OUTPUT = """
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.10.10.1
Host is up (0.015s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1
80/tcp    open  http     Apache httpd 2.4.52
443/tcp   open  ssl/http Apache httpd 2.4.52
445/tcp   open  smb      Samba 4.15.2

Nmap done: 1 IP address (1 host up) scanned in 12.34 seconds
"""

# Mock enum4linux output
MOCK_ENUM4LINUX_OUTPUT = """
=====
|   Users on 10.10.10.1   |
=====
user:[administrator] rid:[0x1f4]
user:[guest] rid:[0x1f5]
user:[jsmith] rid:[0x44f]
"""

# Mock whatweb output
MOCK_WHATWEB_OUTPUT = """
http://10.10.10.1 [200 OK] Apache[2.4.52], Country[RESERVED][ZZ],
HTTPServer[Ubuntu Linux][Apache/2.4.52 (Ubuntu)], IP[10.10.10.1],
Title[Default Page], WordPress[6.1.1]
"""
```

Strategy: Intercept Runner.run Based on Command Content

For modules that call multiple tools, route mock responses based on the command being executed:

```
def mock_runner_side_effect(command, **kwargs):
    """Route mock responses based on command content."""
    cmd_str = " ".join(command) if isinstance(command, (list, tuple)) else command

    if "nmap" in cmd_str:
        return make_run_result(stdout=MOCK_NMAP_OUTPUT)
    elif "enum4linux" in cmd_str:
        return make_run_result(stdout=MOCK_ENUM4LINUX_OUTPUT)
    elif "whatweb" in cmd_str:
        return make_run_result(stdout=MOCK_WHATWEB_OUTPUT)
    elif "wafw00f" in cmd_str:
        return make_run_result(stdout="No WAF detected")
    elif "gobuster" in cmd_str:
        return make_run_result(stdout="")
    elif "nikto" in cmd_str:
        return make_run_result(stdout="")
    else:
        return make_run_result(stdout="")
```

Validating Output Structure

After running a mocked module, verify the output directory:

```
def assert_output_structure(output_dir: Path, module: str):
    """Validate the expected output directory structure."""
    module_dir = output_dir / module

    # Core output files
    assert module_dir.exists(), f"Module directory missing: {module_dir}"
    assert (module_dir / "findings.json").exists(), "findings.json missing"
    assert (module_dir / "findings.md").exists(), "findings.md missing"
    assert (module_dir / "session.md").exists(), "session.md missing"
    assert (module_dir / "commands.log").exists(), "commands.log missing"

    # Validate findings.json is valid JSON
    findings_data = json.loads((module_dir / "findings.json").read_text())
    assert isinstance(findings_data, list), "findings.json should be a JSON array"

    # Validate each finding has required fields
    for finding in findings_data:
        assert "severity" in finding
        assert "confidence" in finding
        assert "description" in finding
        assert "module" in finding
        assert finding["severity"] in ("critical", "high", "medium", "low", "info")
        assert finding["confidence"] in ("confirmed", "high", "medium", "low", "heuristic")
```

Checking Findings Generation

Verify that findings are generated with correct classification:

```
def assert_findings_valid(findings_manager):
    """Validate findings classification rules."""
    for finding in findings_manager.get_all():
        # Heuristic findings must not exceed low severity
        if finding.confidence == "heuristic":
            assert finding.severity in ("low", "info"), \
                f"Heuristic finding has severity {finding.severity}: {finding.description}"

        # All findings must have descriptions
        assert finding.description, f"Finding {finding.id} has empty description"

        # All findings must have a module
        assert finding.module, f"Finding {finding.id} has no module"
```

Test Coverage Areas

Module Initialization

Test that modules initialize correctly with various parameter combinations:

```
def test_network_module_init(tmp_path):
    """Test NetworkModule initializes with valid parameters."""
    with patch("core.runner.Runner.run", return_value=make_run_result()):
        with patch("core.runner.Runner.check_tool", return_value=True):
            from modules.network.network_module import NetworkModule
            module = NetworkModule(
                target="10.10.10.1",
                output_base=str(tmp_path),
                opsec_mode="normal",
                verbose=True,
                dry_run=True,
            )
            assert module.MODULE_NAME == "network"
            assert module.VALID_PHASES == ["discovery", "scanning", "enumeration", "authentication"]
```

Phase Execution

Verify that phases run in order and respect OPSEC:

```
def test_phase_selection(tmp_path):
    """Test that only specified phases execute."""
    with patch("core.runner.Runner.run", return_value=make_run_result(stdout=MOCK_NMAP_OUTPUT)):
        with patch("core.runner.Runner.check_tool", return_value=True):
            from modules.network.network_module import NetworkModule
            module = NetworkModule(
                target="10.10.10.1",
                output_base=str(tmp_path),
                dry_run=True,
            )
            results = module.run(phases=["discovery"])
            assert "discovery" in results.get("phases", {})
            # scanning, enumeration, authentication should not be present
```

Tool Wrapper Calls

Verify that tool wrappers construct correct commands:

```
def test_nmap_command_construction():
    """Test that NmapTool builds correct command lists."""
    from modules.network.tools.nmap import NmapTool
    from core.logger import ReconLogger
    from core.runner import Runner

    logger = ReconLogger(name="test", verbose=False)
    runner = Runner(logger=logger, dry_run=True)
    # Test command construction via dry_run
    result = runner.run(["nmap", "-sS", "--open", "10.10.10.1"])
    assert result.success # dry_run always succeeds
```

Loot Collection

Verify that loot is extracted correctly from parsed results:

```
def test_loot_deduplication():
    """Test that duplicate loot is deduplicated."""
    from core.loot_manager import LootManager

    loot = LootManager()
    loot.add_user("jsmith", "enum4linux", "network")
    loot.add_user("jsmith", "ldapsearch", "network") # Duplicate value

    assert len(loot.get_by_type("user")) == 1 # Deduped
```

Notes Generation

Verify session notes are generated:

```
def test_notes_timeline():
    """Test that notes manager creates proper timeline."""
    from core.notes_manager import NotesManager

    notes = NotesManager(target="10.10.10.1")
    notes.add_phase_start("discovery")
    notes.add_command_note("nmap -sn 10.10.10.1", "1 host up")
    notes.add_finding_note("OpenSSH on port 22")
    notes.add_phase_end("discovery", "1 host found")

    md = notes.to_markdown()
    assert "10.10.10.1" in md
    assert "discovery" in md
    assert "nmap" in md
```

Recommended Fixtures

Mock Tool Outputs

Create a fixtures directory with realistic tool output samples:

```

tests/fixtures/
├── nmap/
│   ├── syn_scan_single_host.txt
│   ├── syn_scan_cidr.txt
│   ├── version_scan.txt
│   └── smb_scripts.txt
├── enum4linux/
│   ├── full_enum.txt
│   └── users_only.txt
├── whatweb/
│   └── wordpress_site.txt
├── nuclei/
│   ├── api_findings.json
│   └── cve_findings.json
├── ffuf/
│   ├── dir_scan.json
│   └── api_fuzz.json
├── bloodhound/
│   └── dc_only.json

```

Load fixtures in tests:

```

import pytest
from pathlib import Path

FIXTURES_DIR = Path(__file__).parent / "fixtures"

@pytest.fixture
def nmap_syn_output():
    return (FIXTURES_DIR / "nmap" / "syn_scan_single_host.txt").read_text()

@pytest.fixture
def enum4linux_output():
    return (FIXTURES_DIR / "enum4linux" / "full_enum.txt").read_text()

```

Sample Target Data

```

@pytest.fixture
def sample_targets():
    return {
        "single_ip": "10.10.10.1",
        "cidr": "10.10.10.0/24",
        "hostname": "dc01.corp.local",
        "web_url": "https://app.target.com",
        "api_url": "https://api.target.com/v1",
    }

```

Configuration Fixtures

The existing `conftest.py` provides `config_dir` with minimal YAML. Extend for specific tests:

```

@pytest.fixture
def full_config_dir(tmp_path):
    """Config directory with full tools.yaml and profiles.yaml."""
    cfg = tmp_path / "config"
    cfg.mkdir()

    # Copy actual config files for realistic testing
    import shutil
    project_root = Path(__file__).parent.parent
    shutil.copy(project_root / "config" / "tools.yaml", cfg / "tools.yaml")
    shutil.copy(project_root / "config" / "profiles.yaml", cfg / "profiles.yaml")

    return cfg

```

Expected Findings

```

@pytest.fixture
def expected_network_findings():
    """Expected findings from a network scan of a typical host."""
    return [
        {"severity": "info", "type": "service", "description_contains": "SSH"},
        {"severity": "info", "type": "service", "description_contains": "HTTP"},
        {"severity": "info", "type": "service", "description_contains": "SMB"},
    ]

```

Expected Assertions

Output Directory Structure

```

def test_full_output_structure(tmp_path):
    """Verify complete output directory is created."""
    output_base = tmp_path / "outputs"

    with patch("core.runner.Runner.run", side_effect=mock_runner_side_effect):
        with patch("core.runner.Runner.check_tool", return_value=True):
            from modules.network.network_module import NetworkModule
            module = NetworkModule(
                target="10.10.10.1",
                output_base=str(output_base),
                dry_run=True,
            )
            module.run()

    target_dir = output_base / "10.10.10.1" / "network"
    assert target_dir.exists()

    # Raw and parsed directories
    assert (target_dir / "raw").exists() or True # raw/ created on tool output write
    # Core files
    assert (target_dir / "session.md").exists()
    assert (target_dir / "commands.log").exists()

```

Findings Files

```
def test_findings_files_created(output_dir):
    """Verify findings are saved in both JSON and Markdown formats."""
    findings_json = output_dir / "findings.json"
    findings_md = output_dir / "findings.md"

    assert findings_json.exists()
    assert findings_md.exists()

    # JSON is valid and parseable
    data = json.loads(findings_json.read_text())
    assert isinstance(data, list)

    # Markdown has expected structure
    md = findings_md.read_text()
    assert "# Security Findings" in md
```

Loot Files

```
def test_loot_file_created(output_dir):
    """Verify loot.json is created with correct structure."""
    loot_path = output_dir / "loot.json"
    assert loot_path.exists()

    data = json.loads(loot_path.read_text())
    assert isinstance(data, list)

    for item in data:
        assert "loot_type" in item
        assert "value" in item
        assert "source" in item
        assert "module" in item
        assert item["loot_type"] in ("credential", "hash", "token", "user", "share", "service")
```

Notes Files

```
def test_session_notes_created(output_dir):
    """Verify session.md is created with timeline entries."""
    session_path = output_dir / "session.md"
    assert session_path.exists()

    content = session_path.read_text()
    assert "# Session Notes" in content
    assert "Target:" in content
    assert "Timeline" in content
```


Findings Content Validation

```
def test_findings_severity_clamping():
    """Verify that heuristic findings are capped at low severity."""
    from core.findings_manager import FindingsManager

    fm = FindingsManager(strict=True)
    finding = fm.add(
        finding_type="vulnerability",
        severity="high",          # Requested high
        confidence="heuristic",  # But heuristic confidence
        target="10.10.10.1",
        module="test",
        description="Test heuristic finding",
    )

    assert finding.severity == "low" # Clamped to low
    assert fm.clamped_count == 1
```

Loot Content Validation

```
def test_loot_credential_structure():
    """Verify credential loot has correct metadata."""
    from core.loot_manager import LootManager

    loot = LootManager()
    loot.add_credential("admin", "password123", "hydra", "network", service="ssh")

    creds = loot.get_credentials()
    assert len(creds) == 1
    assert creds[0]["username"] == "admin"
    assert creds[0]["password"] == "password123"
    assert creds[0]["service"] == "ssh"
```

Example: Complete Integration Test

A full working example testing the network module end-to-end with mocked tools:

```

"""Integration test: Network module end-to-end with mocked tools."""

import json
import sys
from pathlib import Path
from unittest.mock import patch, MagicMock

import pytest

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from core.runner import RunResult

# — Mock data —

MOCK_NMAP_PING = """
Starting Nmap 7.94
Nmap scan report for 10.10.10.1
Host is up (0.010s latency).
Nmap done: 1 IP address (1 host up)
"""

MOCK_NMAP_SCAN = """
Starting Nmap 7.94
Nmap scan report for 10.10.10.1
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1
80/tcp    open  http     Apache httpd 2.4.52
445/tcp   open  smb      Samba 4.15.2
Nmap done: 1 IP address (1 host up) scanned in 5.00 seconds
"""

MOCK_ENUM4LINUX = """
=====
|   Users on 10.10.10.1   |
=====
user:[administrator] rid:[0x1f4]
user:[jsmith] rid:[0x44f]
"""

def _make_result(stdout="", success=True):
    return RunResult(
        command="mock", returncode=0 if success else 1,
        stdout=stdout, stderr="", duration=0.1, success=success,
    )

def _mock_run(command, **kwargs):
    cmd = " ".join(command) if isinstance(command, (list, tuple)) else command
    if "-sn" in cmd:
        return _make_result(stdout=MOCK_NMAP_PING)
    elif "nmap" in cmd:
        return _make_result(stdout=MOCK_NMAP_SCAN)
    elif "enum4linux" in cmd:
        return _make_result(stdout=MOCK_ENUM4LINUX)
    elif "smbclient" in cmd:
        return _make_result(stdout="")
    elif "ldapsearch" in cmd:
        return _make_result(stdout="")
    elif "hydra" in cmd:

```

```

        return _make_result(stdout="")
    return _make_result()

# — Test —————

@pytest.fixture
def output_dir(tmp_path):
    return tmp_path / "outputs"

def test_network_module_integration(output_dir):
    """Full end-to-end test of network module with mocked tools."""
    with patch("core.runner.Runner.run", side_effect=_mock_run):
        with patch("core.runner.Runner.check_tool", return_value=True):
            from modules.network.network_module import NetworkModule

            module = NetworkModule(
                target="10.10.10.1",
                output_base=str(output_dir),
                opsec_mode="normal",
                verbose=False,
                dry_run=False,
                timeout=60,
            )

            results = module.run()

# — Assertions —————

# 1. Module returned results
assert results is not None
assert "phases" in results

# 2. Output directory exists
module_dir = output_dir / "10.10.10.1" / "network"
assert module_dir.exists()

# 3. Session notes created
session = module_dir / "session.md"
assert session.exists()
content = session.read_text()
assert "10.10.10.1" in content

# 4. Commands logged
cmd_log = module_dir / "commands.log"
assert cmd_log.exists()

# 5. Findings saved (if any were generated)
findings_json = module_dir / "findings.json"
if findings_json.exists():
    data = json.loads(findings_json.read_text())
    assert isinstance(data, list)
    for f in data:
        assert "severity" in f
        assert "confidence" in f
        assert f["severity"] in ("critical", "high", "medium", "low", "info")

# 6. Loot saved (if any was collected)
loot_json = module_dir / "loot.json"
if loot_json.exists():
    data = json.loads(loot_json.read_text())
    assert isinstance(data, list)

```

```

    for item in data:
        assert "loot_type" in item
        assert "value" in item

def test_network_module_dry_run(output_dir):
    """Verify dry-run mode doesn't execute commands."""
    from modules.network.network_module import NetworkModule

    with patch("core.runner.Runner.check_tool", return_value=True):
        module = NetworkModule(
            target="10.10.10.1",
            output_base=str(output_dir),
            dry_run=True,
        )
        results = module.run()

    # Dry run should still produce structure
    assert results is not None

def test_network_module_stealth_opsec(output_dir):
    """Verify stealth mode blocks high-noise techniques."""
    with patch("core.runner.Runner.run", side_effect=_mock_run):
        with patch("core.runner.Runner.check_tool", return_value=True):
            from modules.network.network_module import NetworkModule

            module = NetworkModule(
                target="10.10.10.1",
                output_base=str(output_dir),
                opsec_mode="stealth",
                verbose=False,
            )

            results = module.run()

    # Stealth mode should still return results (just fewer)
    assert results is not None

```

Best Practices

Test Isolation

- Use `tmp_path` (pytest built-in) for all output directories — no leftover files
- Patch at the `Runner` level, not individual tool classes — single interception point
- Each test should be independent — no shared state between tests
- Reset singletons and caches if any module uses them

Fixture Management

- Store mock tool outputs in `tests/fixtures/` as text files — not inline strings for large outputs
- Version fixture files alongside code — output format changes should update fixtures
- Use `@pytest.fixture` for common setup (config dirs, output dirs, mock runners)
- The existing `conftest.py` provides `tmp_dir` and `config_dir` fixtures — extend rather than duplicate

Mock Data Realism

- Use actual tool output as mock data — run the tool once, capture output, sanitize
- Include edge cases: empty output, error messages, partial results
- Test with multiple hosts in nmap output, not just single hosts
- Include both successful and failed tool executions

Assertion Completeness

- Assert output **structure** (files exist, directories created)
- Assert output **content** (valid JSON, required fields present)
- Assert **classification rules** (severity clamping, confidence levels)
- Assert **negative cases** (stealth mode blocks techniques, missing tools produce warnings)
- Assert **idempotency** (running twice produces consistent results)

Running Tests

```
# Run all tests
cd /path/to/reconforge
python -m pytest tests/ -v

# Run integration tests only
python -m pytest tests/ -v -k "integration"

# Run with coverage
python -m pytest tests/ --cov=core --cov=modules --cov-report=term-missing

# Run a single test file
python -m pytest tests/core/test_runner.py -v
```